

Lab Assignment # 05

Course Title : AI Assistant Coding
Name of Student : M. Sai Pallavi
Enrollment No. : 2303A54063
Batch No. : 48

Lab 5: Ethical Foundations – Responsible AI Coding Practices

Task Description #1 (Privacy in API Usage)

Task: Use an AI tool to generate a Python program that connects to a weather API.

Prompt: "Generate code to fetch weather data securely without exposing API keys in the code."

Expected Output:

- Original AI code (check if keys are hardcoded).
- Secure version using environment variables.

Output Screenshot:

```
5
6 # Load variables from .env file
7 load_dotenv()
8
9 api_key = os.getenv('WEATHER_API_KEY')
10 city = input("Enter city name: ")
11 base_url = "http://api.openweathermap.org/data/2.5/weather?"
12 complete_url = f"{base_url}q={city}&appid={api_key}&units=metric"
13
14 response = requests.get(complete_url)
15 data = response.json()
16
17 if data.get("cod") == 200:
18     main = data["main"]
19     weather_desc = data["weather"][0]["description"]
20     print(f"City: {city}")
21     print(f"Temperature: {main['temp']}°C")
22     print(f"Weather: {weather_desc}")
23 else:
24     print("Error:", data.get("message", "City Not Found"))
25
26 # Ensure to create a .env file with the line: WEATHER_API_KEY=your_api_key_here
27
28
29
30
31
32
33
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/weather.py"
Traceback (most recent call last):
```

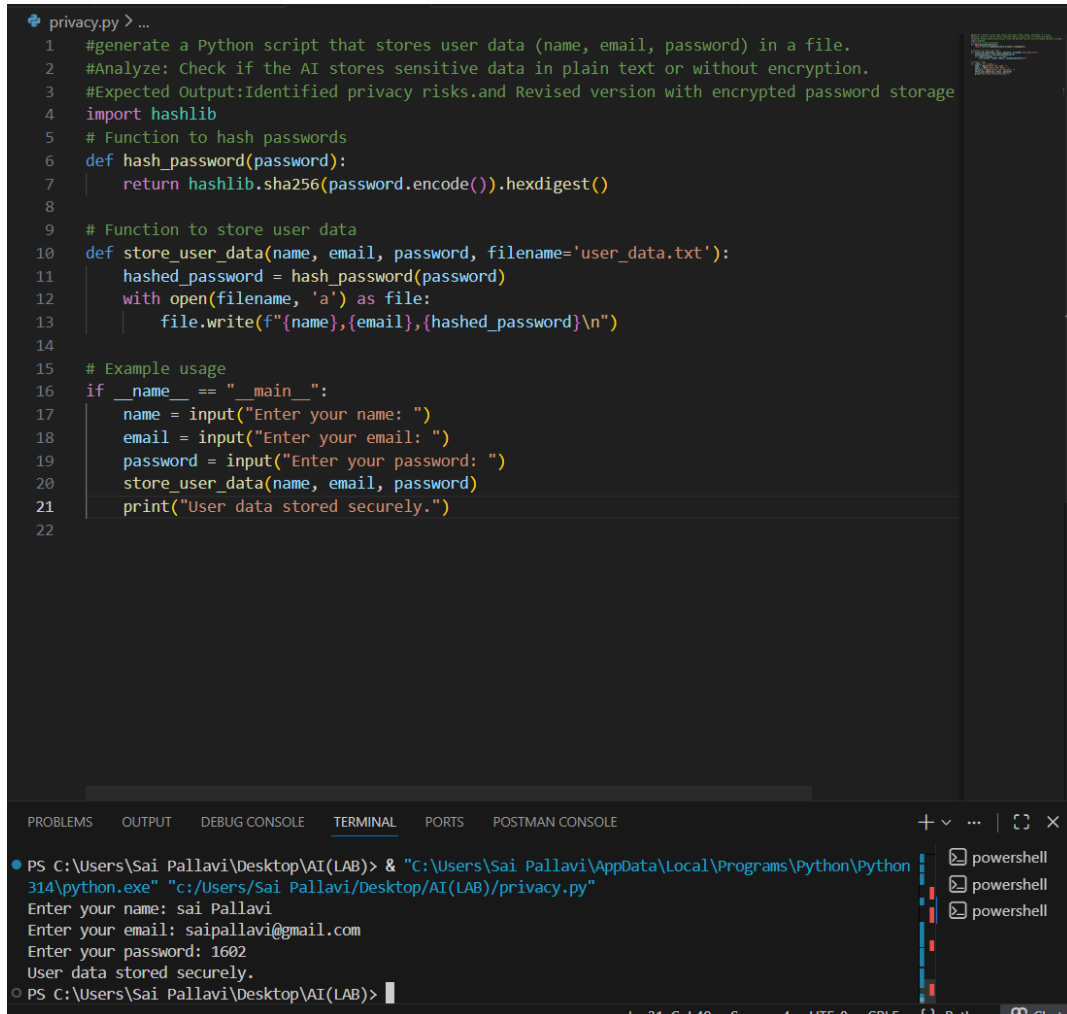
Task Description #2 (Privacy & Security in File Handling)

Task: Use an AI tool to generate a Python script that stores user data (name, email, password) in a file. **Analyze:** Check if the AI stores sensitive data in plain text or without encryption.

Expected Output:

- Identified privacy risks.
- Revised version with encrypted password storage (e.g., hashing).

Output Screenshot:



```
1 #generate a Python script that stores user data (name, email, password) in a file.
2 #Analyze: Check if the AI stores sensitive data in plain text or without encryption.
3 #Expected Output:Identified privacy risks.and Revised version with encrypted password storage
4 import hashlib
5 # Function to hash passwords
6 def hash_password(password):
7     return hashlib.sha256(password.encode()).hexdigest()
8
9 # Function to store user data
10 def store_user_data(name, email, password, filename='user_data.txt'):
11     hashed_password = hash_password(password)
12     with open(filename, 'a') as file:
13         file.write(f"{name},{email},{hashed_password}\n")
14
15 # Example usage
16 if __name__ == "__main__":
17     name = input("Enter your name: ")
18     email = input("Enter your email: ")
19     password = input("Enter your password: ")
20     store_user_data(name, email, password)
21     print("User data stored securely.")
22
```

PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/privacy.py"

Enter your name: sai Pallavi
Enter your email: saipallavi@gmail.com
Enter your password: 1602
User data stored securely.

PS C:\Users\Sai Pallavi\Desktop\AI(LAB)>

Task Description #3 (Transparency in Algorithm Design)

Objective: Use AI to generate an Armstrong number checking function with comments and explanations.

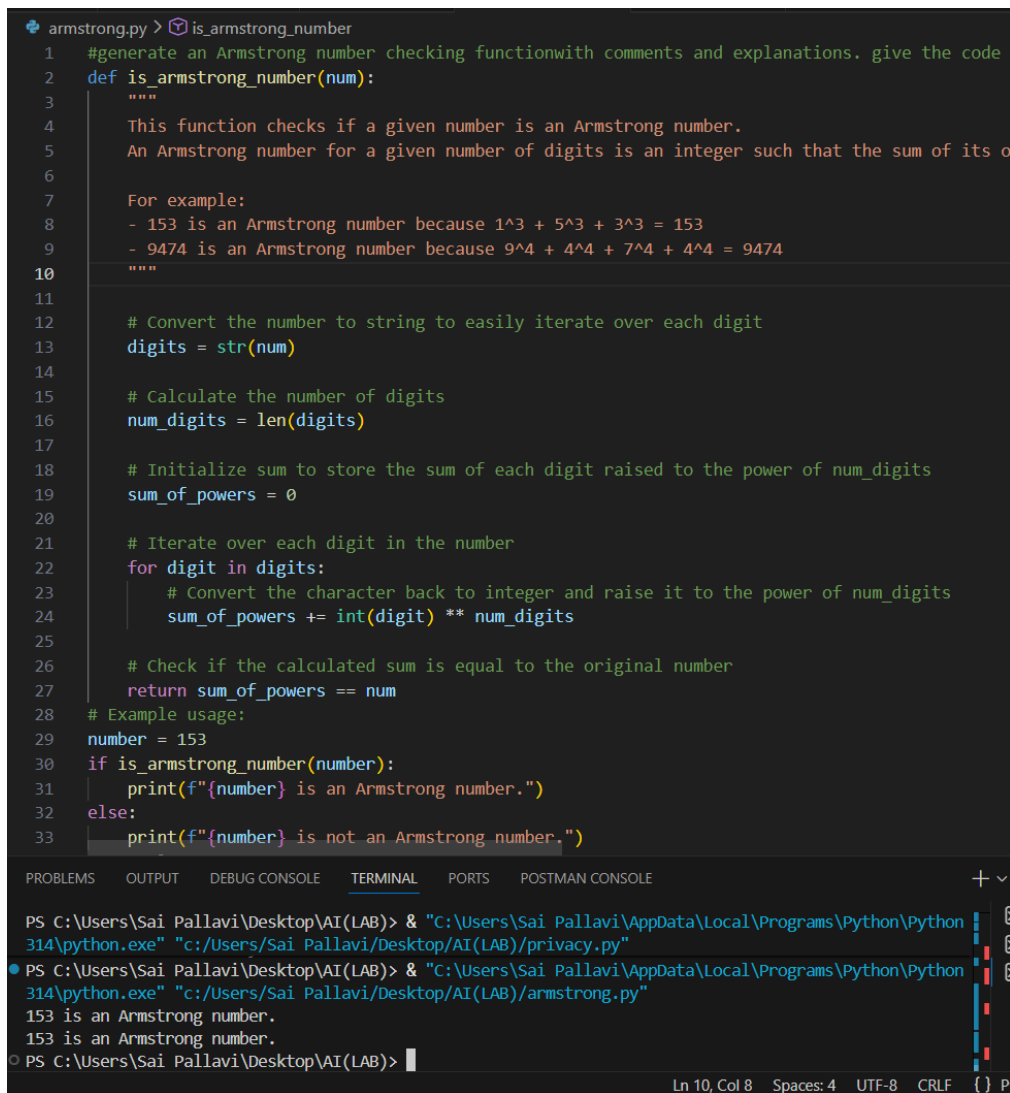
Instructions:

1. Ask AI to explain the code line-by-line.
2. Compare the explanation with code functionality.

Expected Output:

- Transparent, commented code.
- Correct, easy-to-understand explanation.

Output Screenshot:



```
armstrong.py > is_armstrong_number
1  #generate an Armstrong number checking function with comments and explanations. give the code
2  def is_armstrong_number(num):
3      """
4      This function checks if a given number is an Armstrong number.
5      An Armstrong number for a given number of digits is an integer such that the sum of its own
6      digits to the power of the number of digits is equal to the number itself.
7
8      For example:
9      - 153 is an Armstrong number because 1^3 + 5^3 + 3^3 = 153
10     - 9474 is an Armstrong number because 9^4 + 4^4 + 7^4 + 4^4 = 9474
11
12     """
13
14     # Convert the number to string to easily iterate over each digit
15     digits = str(num)
16
17     # Calculate the number of digits
18     num_digits = len(digits)
19
20     # Initialize sum to store the sum of each digit raised to the power of num_digits
21     sum_of_powers = 0
22
23     # Iterate over each digit in the number
24     for digit in digits:
25         # Convert the character back to integer and raise it to the power of num_digits
26         sum_of_powers += int(digit) ** num_digits
27
28     # Check if the calculated sum is equal to the original number
29     return sum_of_powers == num
30
31 # Example usage:
32 number = 153
33 if is_armstrong_number(number):
34     print(f"{number} is an Armstrong number.")
35 else:
36     print(f"{number} is not an Armstrong number.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/privacy.py"
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/armstrong.py"
153 is an Armstrong number.
153 is an Armstrong number.
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)>
```

Ln 10, Col 8 Spaces: 4 UTF-8 CRLF {} Py

Task Description #4 (Transparency in Algorithm Comparison)

Task: Use AI to implement two sorting algorithms (e.g., QuickSort and BubbleSort).

Prompt: "Generate Python code for QuickSort and BubbleSort, and include comments explaining step-by-step how each works and where they differ."

Expected Output:

- Code for both algorithms.
- Transparent, comparative explanation of their logic and efficiency.

Output Screenshot:

```

1 #Generate Python code for QuickSort and BubbleSort, and include comments explaining step-by-st
2 # QuickSort implementation
3 def quicksort(arr):
4     """
5     QuickSort is a divide-and-conquer algorithm that works by selecting a 'pivot' element from
6     and partitioning the other elements into two sub-arrays according to whether they are less
7     The sub-arrays are then sorted recursively.
8     """
9     if len(arr) <= 1:
10         return arr # Base case: arrays with 0 or 1 element are already sorted
11
12     pivot = arr[len(arr) // 2] # Choose the middle element as the pivot
13     left = [x for x in arr if x < pivot] # Elements less than the pivot
14     middle = [x for x in arr if x == pivot] # Elements equal to the pivot
15     right = [x for x in arr if x > pivot] # Elements greater than the pivot
16
17     # Recursively apply quicksort to left and right sub-arrays and combine results
18     return quicksort(left) + middle + quicksort(right)
19
20 # BubbleSort implementation
21 def bubblesort(arr):
22     """
23     BubbleSort is a simple sorting algorithm that repeatedly steps through the list,
24     compares adjacent elements and swaps them if they are in the wrong order.
25     The process is repeated until the list is sorted.
26     """
27     n = len(arr)
28     # Traverse through all array elements
29     for i in range(n):
30         swapped = False # Flag to check if any swapping occurred
31         # Last i elements are already sorted, no need to check them
32         for j in range(0, n-i-1):
33             # Compare adjacent elements
34             if arr[j] > arr[j+1]:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
 PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
 314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/armstrong.py"
 PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
 314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/sorting.py"
 Original array: [64, 34, 25, 12, 22, 11, 90]
 Sorted array using QuickSort: [11, 12, 22, 25, 34, 64, 90]
 Sorted array using BubbleSort: [11, 12, 22, 25, 34, 64, 90]
 PS C:\Users\Sai Pallavi\Desktop\AI(LAB)>

Key Differences

- QuickSort is much faster in practice because it reduces the problem size at each step by partitioning the array.
- BubbleSort is conceptually simple but inefficient, as it repeatedly scans the array and swaps elements, leading to quadratic time complexity.
- QuickSort is widely used in real-world applications, while BubbleSort is mostly used for educational purposes to demonstrate

◆ Efficiency Comparison

	$O(n \log n)$	$O(n)$
	$O(n \log n)$	$O(n^2)$
	$O(n^2)$	$O(n^2)$
	$O(\log n)$	$O(1)$

◆ Transparent Takeaways

- QuickSort is efficient and scalable:
It reduces the sorting problem size at each step, making it suitable for large datasets. Its recursive nature leverages divide-and-conquer, which is why it's widely used in practice.
- BubbleSort is simple but inefficient:
It's easy to understand and implement, but its repeated passes make it extremely slow for large inputs. It's mainly used for teaching sorting concepts, not real-world applications.
- Key Difference in Logic:
QuickSort *divides the array into smaller problems*, while BubbleSort *repeatedly pushes larger elements to the end*.
This difference in strategy explains why QuickSort achieves logarithmic efficiency while BubbleSort suffers quadratic inefficiency

Task Description #5 (Transparency in AI Recommendations)

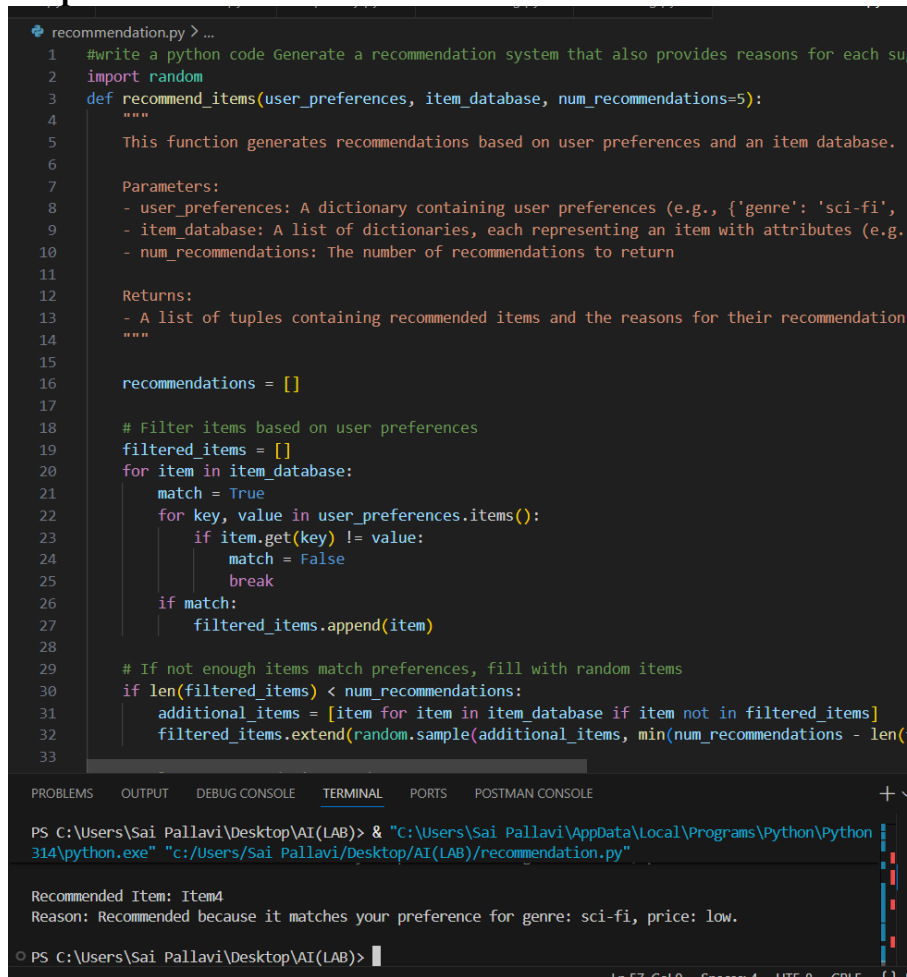
Task: Use AI to create a product recommendation system.

Prompt: "Generate a recommendation system that also provides reasons for each suggestion."

Expected Output:

- Code with explainable recommendations.
- Evaluation of whether explanations are understandable.

Output Screenshot:



```
recommendation.py > ...
1  #write a python code Generate a recommendation system that also provides reasons for each su
2  import random
3  def recommend_items(user_preferences, item_database, num_recommendations=5):
4      """
5      This function generates recommendations based on user preferences and an item database.
6
7      Parameters:
8      - user_preferences: A dictionary containing user preferences (e.g., {'genre': 'sci-fi',
9      - item_database: A list of dictionaries, each representing an item with attributes (e.g.
10     - num_recommendations: The number of recommendations to return
11
12     Returns:
13     - A list of tuples containing recommended items and the reasons for their recommendation
14     """
15
16     recommendations = []
17
18     # Filter items based on user preferences
19     filtered_items = []
20     for item in item_database:
21         match = True
22         for key, value in user_preferences.items():
23             if item.get(key) != value:
24                 match = False
25                 break
26         if match:
27             filtered_items.append(item)
28
29     # If not enough items match preferences, fill with random items
30     if len(filtered_items) < num_recommendations:
31         additional_items = [item for item in item_database if item not in filtered_items]
32         filtered_items.extend(random.sample(additional_items, min(num_recommendations - len(
33
Terminal Output:
PS C:\Users\Sai Pallavi\Desktop\AI(LAB)> & "C:\Users\Sai Pallavi\AppData\Local\Programs\Python\Python
314\python.exe" "c:/Users/Sai Pallavi/Desktop/AI(LAB)/recommendation.py"

Recommended Item: Item4
Reason: Recommended because it matches your preference for genre: sci-fi, price: low.

PS C:\Users\Sai Pallavi\Desktop\AI(LAB)>
```

Evaluation of Explanations

- **Understandability:**
The explanations are clear and human-friendly. They explicitly state:
 - The product name.
 - Why it matches the user's interest (category alignment).
 - Why it fits the budget.
 - Why it's considered good (rating + popularity).
- **Transparency:**
Users can see the criteria used (interest, budget, rating, popularity). This avoids the "black box" problem of AI recommendations.
- **Strengths:**
 - Easy to read and interpret.
 - Shows multiple factors (budget, rating, popularity).
 - Provides confidence in why the product was chosen.
- **Possible Improvements:**

- Could add trade-offs (e.g., “slightly higher price but better rating”).
- Could personalize further (e.g., “similar to items you liked before”).